Le squelette d'un jeu | exemples/ecrans.cpp | 808 octets de programme | tout est mesuré, rien n'est recopié

Passer d'un écran à l'autre

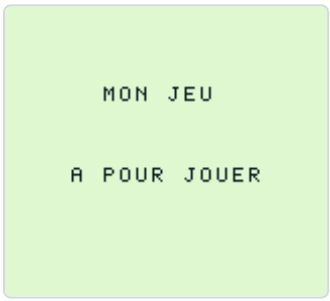
Un titre, une partie, une fin — et le bouton qui mène de l'un à l'autre.

C'est le squelette de n'importe quel jeu : un écran d'accueil, la partie, l'écran de fin. Tout tient en trois pièces, et il n'y en a pas d'autres.

1. **Une variable dit où l'on est.** Un octet, `scene`, qui vaut `TITRE`, `JEU` ou `FIN`. Le reste du programme ne fait que la lire.
2. **Un seul endroit change d'écran.** `allerA()` éteint, efface, redessine, rallume. Si chaque bouton faisait son propre ménage, l'un finirait par oublier d'effacer, et deux écrans se mélangeraient.
3. **On ne redessine qu'au changement.** Redessiner à chaque image coûterait 360 cases soixante fois par seconde, pour rien : l'écran ne bouge pas tant que la scène ne change pas.

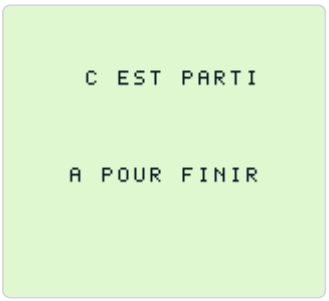
1. Ce que ça donne

Les quatre images ci-dessous sont l'écran de la console émulée, photographié après chaque appui sur `A`. Le quatrième appui ramène au titre : la roue se referme.



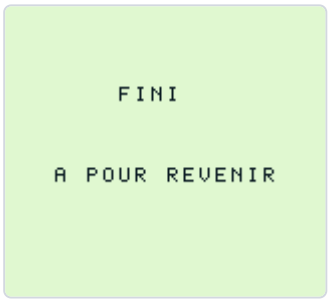
TITRE

au démarrage



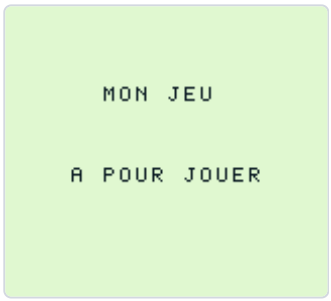
JEU

après un appui sur A



FIN

après un deuxième appui



TITRE

après un troisième appui

TITRE — A → JEU — A → FIN — A → TITRE

2. Le programme, morceau par morceau

LES VALEURS DE LA SCÈNE, ET LA SCÈNE ELLE-MÊME

```
uint8_t TITRE = 0;
uint8_t JEU = 1;
uint8_t FIN = 2;
uint8_t scene = 0;
uint8_t a = 0;
uint8_t aAvant = 0;
uint8_t x = 0;
uint8_t y = 0;
```

Trois noms plutôt que trois nombres : `scene == JEU` se relit, `scene == 1` demande de se souvenir. `aAvant` retient l'état du bouton à l'image précédente — on verra au chapitre 4 pourquoi il est indispensable.

EFFACER() — LES 20 × 18 CASES DE L'ÉCRAN

```
effacer();   // dans allerA(), avant de dessiner le nouvel écran
```

effacer() est une fonction de la console (demandée en haut par sa ligne #include) : sans argument, elle vide tout l'écran. Sans cet effacement, « A POUR JOUER » resterait sous « A POUR FINIR », les deux textes mélangés à l'écran.

UN DESSIN PAR ÉCRAN

```
void dessinerTitre() {
  texte(6, 5, "MON JEU");
  texte(4, 10, "A POUR JOUER");
}

void dessinerJeu() {
  texte(5, 4, "C EST PARTI");
  texte(4, 10, "A POUR FINIR");
}

void dessinerFin() {
  texte(7, 5, "FINI");
  texte(3, 10, "A POUR REVENIR");
}
```

Chacune ne sait que dessiner. Aucune ne décide de rien, aucune n'efface : c'est ce qui permet de les appeler depuis un seul endroit.

ALLERA — LE SEUL ENDROIT QUI CHANGE D'ÉCRAN

```
void allerA() {
  ecran(0);
  effacer();
  if (scene == TITRE) {
    dessinerTitre();
  }
  if (scene == JEU) {
    dessinerJeu();
  }
  if (scene == FIN) {
    dessinerFin();
  }
  ecran(1);
}
```

L'ordre compte. `ecran(0)` éteint l'affichage, on fait le ménage à l'abri, `ecran(1)` rallume. Repeindre 360 cases prend bien plus de temps qu'une image n'en contient : écran allumé, on verrait le ménage se faire.

MAIN — LA BOUCLE, ET LE BOUTON

```
int main() {
  scene = TITRE;
  allerA();
  while (true) {
    image();
    a = bouton(A);
    if (a == 1) {
      if (aAvant == 0) {
        if (scene == TITRE) {
          scene = JEU;
        } else {
          if (scene == JEU) {
            scene = FIN;
          } else {
            scene = TITRE;
          }
        }
        allerA();
      }
    }
    aAvant = a;
  }

  return 0;
}
```

`image()` attend la fin de l'image en cours. En dehors du changement de scène, cette boucle ne fait rien d'autre que lire un bouton : c'est normal, l'écran est déjà peint.

3. Pourquoi `ecran(0)` avant d'effacer

Une Game Boy affiche soixante images par seconde. Entre deux images, elle laisse un court instant — le *VBlank* — pendant lequel la mémoire vidéo est libre. Poser 360 tuiles demande beaucoup plus que cet instant.

> **Le piège :** si on efface écran allumé, le balayage passe pendant le ménage et affiche un mélange des deux écrans — un scintillement, ou une bande de l'ancien texte qui survit une fraction de seconde. Éteindre l'écran suspend le balayage : plus rien ne s'affiche pendant qu'on travaille, et tout apparaît d'un coup au rallumage.

4. Le détecteur de front

C'est le point qui casse le plus souvent, et il ne se voit pas dans le code :

```
a = bouton(A);
if (a == 1) {
  if (aAvant == 0) {   // ← le front : A vient tout juste d'être pressé
    ...
    allerA();
  }
}
aAvant = a;            // ← on retient l'état pour l'image suivante
```

`bouton(A)` ne dit pas « A vient d'être pressé » : il dit « A est enfoncé *maintenant* ». Un appui humain dure une dizaine d'images. Sans `aAvant`, la scène avancerait dix fois — on traverserait les trois écrans et on retomberait sur le titre avant d'avoir eu le temps de lire quoi que ce soit.

La comparaison `a == 1 && aAvant == 0` n'est vraie qu'à **une seule image** : celle où le bouton passe de relâché à enfoncé. Un appui, une transition, quelle que soit sa durée.

5. Ce que la console a répondu

Ces trois lignes ne sont pas une promesse : la cartouche a été chargée dans l'émulateur, le bouton pressé, et `scene` relue dans la mémoire de la console.

✓ au moment de l'appui, la scène avance d'un cran
scene = 1

✓ A maintenu trois secondes : plus rien ne bouge
scene = 1

✓ relâché puis repressé : la scène avance encore
scene = 2

6. Pour t'en servir

Ajouter un écran tient en trois gestes : un nom de plus dans les valeurs de `scene`, une fonction `dessinerX()`, et un `if` de plus dans `allerA()`. Rien d'autre du programme ne change.

> `exemples/ecrans.cpp` — le programme entier, 77 lignes.
> `exemples/decoupe/` — le même, réparti sur cinq fichiers, pour voir `#include` à l'œuvre.
> `livrets/lecon-18-passer-d-un-ecran-a-l-autre.pdf` — la même idée écrite avec `enum` et `switch`.

Si tu débutes : le programme entier, ligne par ligne

Chaque ligne du programme, puis ce qu'elle veut dire en français. Les mots techniques sont expliqués juste en dessous, et dans le lexique à la fin.

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **bouton()** — Une fonction de la console qui dit si un bouton de la manette est enfoncé : 1 oui, 0 non. Les boutons : A, B, HAUT, BAS, GAUCHE, DROITE, START, SELECT.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **else (sinon)** — Ce que l'on fait quand la condition du « if » est fausse.
- **fonction** — Un groupe d'instructions qui a un nom, qu'on lance en écrivant son nom et des parenthèses. Celles de la console (texte, poser…) sont toutes prêtes ; on peut écrire les siennes.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **return** — Termine la fonction et rend une valeur à celui qui l'a appelée.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.

- **void** — Devant une fonction : elle ne rend rien, elle agit seulement.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// Porté depuis ecrans.js par « node porter.mjs ».`	Un commentaire : une phrase pour celui qui lit (« Porté depuis ecrans.js par « node porter.mjs ». »). La console l'ignore.
`// Le portage est littéral : il donne un programme juste, non un programme`	Un commentaire : une phrase pour celui qui lit (« Le portage est littéral : il donne un programme juste, non un programme »). La console l'ignore.
`// idiomatique. Les variables globales qui servaient d'arguments faute de`	Un commentaire : une phrase pour celui qui lit (« idiomatique. Les variables globales qui servaient d'arguments faute de »). La console l'ignore.
`// mieux méritent de redevenir des arguments, et les portées de reprendre`	Un commentaire : une phrase pour celui qui lit (« mieux méritent de redevenir des arguments, et les portées de reprendre »). La console l'ignore.
`// leur place — c'est là tout l'intérêt d'être passé au C++.`	Un commentaire : une phrase pour celui qui lit (« leur place — c'est là tout l'intérêt d'être passé au C++. »). La console l'ignore.
`#include <texte>  // écrit un texte à l'écran`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un texte à l'écran ».)
`#include <ecran>  // éteint ou rallume l'écran`	Demande à la console « ecran » (ce que c'est : éteint ou rallume l'écran). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « éteint ou rallume l'écran ».)
`#include <effacer>  // efface des cases, ou tout le fond`	Demande à la console « effacer » (ce que c'est : efface des cases, ou tout le fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « efface des cases, ou tout le fond ».)
`#include <bouton>  // lit un bouton de la manette`	Demande à la console « bouton » (ce que c'est : lit un bouton de la manette). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « lit un bouton de la manette ».)
`uint8_t TITRE = 0;`	Une variable : une boîte appelée TITRE, qui retient un nombre de 0 à 255, et qui commence à 0.
`uint8_t JEU = 1;`	Une variable : une boîte appelée JEU, qui retient un nombre de 0 à 255, et qui commence à 1.
`uint8_t FIN = 2;`	Une variable : une boîte appelée FIN, qui retient un nombre de 0 à 255, et qui commence à 2.
`uint8_t scene = 0;`	Une variable : une boîte appelée scene, qui retient un nombre de 0 à 255, et qui commence à 0.
`uint8_t a = 0;`	Une variable : une boîte appelée a, qui retient un nombre de 0 à 255, et qui commence à 0.
`uint8_t aAvant = 0;`	Une variable : une boîte appelée aAvant, qui retient un nombre de 0 à 255, et qui commence à 0.
`uint8_t x = 0;`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0.
`uint8_t y = 0;`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 0.

`void dessinerTitre() {`	Le début d'une fonction à nous, appelée dessinerTitre : un groupe d'instructions qu'on lancera en écrivant dessinerTitre(…). Elle ne rend rien (void).
`texte(6, 5, "MON JEU");`	Appel de la fonction texte de la console, avec les arguments 6, 5, "MON JEU". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`texte(4, 10, "A POUR JOUER");`	Appel de la fonction texte de la console, avec les arguments 4, 10, "A POUR JOUER". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`void dessinerJeu() {`	Le début d'une fonction à nous, appelée dessinerJeu : un groupe d'instructions qu'on lancera en écrivant dessinerJeu(…). Elle ne rend rien (void).
`texte(5, 4, "C EST PARTI");`	Appel de la fonction texte de la console, avec les arguments 5, 4, "C EST PARTI". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`texte(4, 10, "A POUR FINIR");`	Appel de la fonction texte de la console, avec les arguments 4, 10, "A POUR FINIR". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`void dessinerFin() {`	Le début d'une fonction à nous, appelée dessinerFin : un groupe d'instructions qu'on lancera en écrivant dessinerFin(…). Elle ne rend rien (void).
`texte(7, 5, "FINI");`	Appel de la fonction texte de la console, avec les arguments 7, 5, "FINI". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`texte(3, 10, "A POUR REVENIR");`	Appel de la fonction texte de la console, avec les arguments 3, 10, "A POUR REVENIR". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`void allerA() {`	Le début d'une fonction à nous, appelée allerA : un groupe d'instructions qu'on lancera en écrivant allerA(…). Elle ne rend rien (void).
`ecran(0);`	Appel de la fonction ecran de la console, avec l'argument 0. Ce qu'elle fait : éteint (0) ou rallume (1) l'écran, le temps d'un gros redessin.
`effacer();`	Appel de la fonction effacer de la console. Ce qu'elle fait : efface un texte à l'écran : « quoi » est le texte lui-même, ou un nombre de cases. Avec un Mot : effacer(SALUT).
`if (scene == TITRE) {`	Une décision : si scene est égal à TITRE, on fait ce qui est entre les accolades. Sinon, on passe.
`dessinerTitre();`	Lance la fonction dessinerTitre : ses instructions s'exécutent, puis on revient à la ligne suivante.

`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`if (scene == JEU) {`	Une décision : si scene est égal à JEU, on fait ce qui est entre les accolades. Sinon, on passe.
`dessinerJeu();`	Lance la fonction dessinerJeu : ses instructions s'exécutent, puis on revient à la ligne suivante.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`if (scene == FIN) {`	Une décision : si scene est égal à FIN, on fait ce qui est entre les accolades. Sinon, on passe.
`dessinerFin();`	Lance la fonction dessinerFin : ses instructions s'exécutent, puis on revient à la ligne suivante.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`ecran(1);`	Appel de la fonction ecran de la console, avec l'argument 1. Ce qu'elle fait : éteint (0) ou rallume (1) l'écran, le temps d'un gros redessin.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`scene = TITRE;`	scene prend une nouvelle valeur : TITRE.
`allerA();`	Lance la fonction allerA : ses instructions s'exécutent, puis on revient à la ligne suivante.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`a = bouton(A);`	a prend une nouvelle valeur : bouton(A).
`if (a == 1) {`	Une décision : si a est égal à 1, on fait ce qui est entre les accolades. Sinon, on passe.
`if (aAvant == 0) {`	Une décision : si aAvant est égal à 0, on fait ce qui est entre les accolades. Sinon, on passe.
`if (scene == TITRE) {`	Une décision : si scene est égal à TITRE, on fait ce qui est entre les accolades. Sinon, on passe.
`scene = JEU;`	scene prend une nouvelle valeur : JEU.
`} else {`	Sinon (si la condition d'avant était fausse), on fait le bloc qui suit.
`if (scene == JEU) {`	Une décision : si scene est égal à JEU, on fait ce qui est entre les accolades. Sinon, on passe.
`scene = FIN;`	scene prend une nouvelle valeur : FIN.
`} else {`	Sinon (si la condition d'avant était fausse), on fait le bloc qui suit.
`scene = TITRE;`	scene prend une nouvelle valeur : TITRE.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`allerA();`	Lance la fonction allerA : ses instructions s'exécutent, puis on revient à la ligne suivante.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

`}`	L’accolade fermante : la fin du bloc ouvert plus haut par « { ».
`aAvant = a;`	aAvant prend une nouvelle valeur : a.
`}`	L’accolade fermante : la fin du bloc ouvert plus haut par « { ».
`return 0;`	La fonction s’arrête ici et rend 0 à celui qui l’a appelée.
`}`	L’accolade fermante : la fin du bloc ouvert plus haut par « { ».

Le lexique : chaque mot, expliqué simplement

Les 55 mots techniques qu'on rencontre dans ce document, par ordre alphabétique.

accolades { }	Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
affectation (=)	« x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
argument	Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
bloc	Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
booléen (vrai / faux)	Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
boucle	Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
boucle du jeu	« while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
bouton()	Une fonction de la console qui dit si un bouton de la manette est enfoncé : 1 oui, 0 non. Les boutons : A, B, HAUT, BAS, GAUCHE, DROITE, START, SELECT.
break	Sort tout de suite de la boucle (ou du « switch ») où l'on est.
C++	Le langage dans lequel ces programmes sont écrits. Le compilateur de l'atelier en comprend une partie, adaptée à la Game Boy.
cartouche	Le boîtier qui contient un jeu ; ici, le fichier .gb (ou .gbc en couleur) que le compilateur fabrique : 32 768 octets.
chaîne de caractères (texte)	Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
commentaire (//)	Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
compilateur	Le traducteur qui transforme le texte du programme en octets que la console comprend, et qui signale les erreurs.
condition	Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
console	La machine qui joue : ici, la Game Boy (ou Game Boy Color), imitée dans le navigateur par l'émulateur.
constante (const)	Une valeur qui a un nom et ne change jamais : « const uint8_t VIES = 3; ».
continue	Passe directement au tour suivant de la boucle.
coordonnées	Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.

couleur, palette	En couleur (Game Boy Color), chaque dessin a quatre teintes ; une palette dit quelles couleurs. couleurFond() les règle pour le décor, couleurLutin() pour les lutins.
défilement	Le décor qui glisse quand le héros avance : defiler(x, y) décale tout le fond.
else (sinon)	Ce que l'on fait quand la condition du « if » est fausse.
émulateur	Un programme qui imite une machine : ici, une Game Boy dans le navigateur.
fonction	Un groupe d'instructions qui a un nom, qu'on lance en écrivant son nom et des parenthèses. Celles de la console (texte, poser…) sont toutes prêtes ; on peut écrire les siennes.
for	Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
hasard()	Une fonction de la console qui rend un nombre au hasard, de 0 à 255.
if (si)	Teste une condition : si elle est vraie, on fait le bloc qui suit.
image()	Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
#include	En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
incrémenter (++)	« x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
indice	Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
instruction	Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
lutin (sprite)	Un dessin qui bouge au pixel près, par-dessus le décor : le héros, un ennemi. sprite() en place un de 8 × 8, sprite16() un de 16 × 16. La console en a 40.
main	« int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
octet	Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
opérateurs de calcul	+ plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
opérateurs de comparaison	== égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
opérateurs logiques	&& « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
paramètre	Dans la définition d'une fonction, la boîte qui reçoit une valeur donnée à l'appel : dans « void f(uint8_t x) », x est un paramètre.
Perso	Un dessin de 16 × 16 pixels (ou 32 × 32) : quatre tuiles en carré (ou seize). « Perso HEROS = { … }; ».
pixel	Un tout petit carré de couleur ; l'écran en a 160 de large et 144 de haut.
point-virgule (;)	Il termine une instruction, comme le point termine une phrase.

poser()	Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
programme	La liste d'instructions qu'on écrit, et que le compilateur traduit en cartouche.
return	Termine la fonction et rend une valeur à celui qui l'a appelée.
struct	Un modèle qui regroupe plusieurs boîtes sous un nom : « struct Ennemi { uint8_t x; uint8_t y; }; ».
switch / case	Choisir entre plusieurs cas selon la valeur d'une variable, au lieu d'une suite de « if ».
tableau	Une rangée de boîtes sous un même nom : « uint8_t cases[10]; » donne cases[0] à cases[9].
taille (agrandir)	Un nombre en dernier argument de sprite, sprite16, sprite32, poser… : le dessin, écrit une fois, montré autant de fois plus grand.
texte()	Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
Tuile	Un dessin de 8 × 8 pixels, écrit rangée par rangée avec des signes (. - + #). Le décor en est fait.
uint8_t	Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
variable	Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
void	Devant une fonction : elle ne rend rien, elle agit seulement.
while (tant que)	Répète un bloc tant qu'une condition est vraie.

Fiche produite par `node fiche-ecrans.mjs` — le programme y est compilé, exécuté et photographié à chaque impression. Elle ne peut pas vieillir sans qu'on le voie.